

Bengio 2003: Fixed-Context Language Models

The simplest neural language model. Look at the last N tokens, embed each through a shared lookup table, concatenate the embeddings, and feed the concatenated vector through an MLP that outputs a distribution over the next token. No recurrence, no attention, no fancy machinery. This is the architecture Bengio et al. introduced in *A Neural Probabilistic Language Model* (2003), and it remained the dominant neural LM for several years before being displaced first by RNNs and then by Transformers.

For the inductive-bias series, this post serves a specific purpose: it shows what you give up and what you gain when you replace an RNN's unbounded hidden state with a fixed window of size N . The model is faster to train (no BPTT), easier to parallelize (no sequential dependencies between training examples), and free of vanishing-gradient issues (no long Jacobian product). What it cannot do is use information from more than N tokens back. It is the Markov assumption of order N , made architectural.

1. The architecture

Three building blocks:

1. **An embedding table** E of shape $V \times d$, where V is the vocabulary size and d is the embedding dimension. For each token id c , the row E_c is a learned dense vector of length d . Embeddings are *shared across positions*: the embedding of “the” is the same whether “the” appears at position 1 or position 5 in the context.
2. **A concatenation step.** Given a context of N token ids $c_{t-N+1}, \dots, c_t$, look up the embeddings $E_{c_{t-N+1}}, \dots, E_{c_t}$ and concatenate them into a single vector $\mathbf{x}$ of length Nd . This vector encodes both *what* tokens are present (through the embeddings) and *where* each one sits in the window (through its position in the concatenation).
3. **A small MLP** that maps $\mathbf{x}$ to logits over the vocabulary:

$$\mathbf{h} = \tanh(W_1 \mathbf{x} + \mathbf{b}_1), \quad \mathbf{z} = W_2 \mathbf{h} + \mathbf{b}_2.$$

Softmax of $\mathbf{z}$ gives the predicted distribution over the next token, and `SoftmaxCrossEntropy` is the loss.

That is the whole model. In `scratchnn` vocabulary it is just a `Network`, with no custom training loop:

```
net = Network([
    EmbedConcat(vocab_size, embed_dim, context_size),
    Linear(context_size * embed_dim, hidden_size),
    Tanh(),
    Linear(hidden_size, vocab_size),
], SoftmaxCrossEntropy())
```

`EmbedConcat` wraps an `Embedding` lookup table plus the concatenation step: forward takes a `list[int]` of length `context_size`, looks up each id in the embedding, and returns a flat `list[float]` of length `context_size * embed_dim`. From the downstream layers' point of view the input is just a vector, so the rest of the library composes without change. Training is

`net.fit(X, Y, ...)` where each `X[i]` is a context (`list[int]`) and each `Y[i]` is the next token id.

2. The Embedding layer

An **Embedding** is mathematically a **Linear** layer applied to a one-hot encoded input, but the multiply-by-zero is skipped through direct indexing. The forward returns a copy of the relevant row; the backward accumulates the upstream gradient into that row.

```
class Embedding(Layer):
    def forward(self, token_id):
        self.cache.append(token_id)
        return list(self.weights[token_id])

    def backward(self, grad):
        token_id = self.cache.pop()
        for k in range(len(grad)):
            self.dweights[token_id][k] += grad[k]
        return None # discrete input, no upstream gradient
```

The lookup view and the one-hot view are mathematically equivalent. Real frameworks (PyTorch’s `nn.Embedding`, etc.) implement the lookup form for efficiency; the math does not change. The weight matrix of the embedding is the matrix you would have used in a **Linear** layer on one-hot inputs.

Because the same **Embedding** is called once per context position in a single forward pass, the layer maintains an internal LIFO cache of which token ids were used. Backward pops the cache and accumulates each gradient into the matching row.

3. The inductive biases

The Bengio LM has three architectural commitments, each one a deliberate bet:

- **Markov of order N .** The model assumes the next token depends only on the previous N tokens. Anything older is forgotten by construction. Choose N too small and the model cannot learn long-range agreement; choose N too large and the MLP head grows linearly in N and the model overfits.
- **Shared embeddings.** Every token has one embedding vector, independent of where in the context it appears. This is weight sharing across the position axis, applied *only* to the embedding layer. The same lookup applies at slot 1, slot 2, ..., slot N .
- **Position-dependent MLP head.** The concatenation step is the crucial choice. By placing the embedding of position 1 in slots 1, ..., d of $\mathbf{x}$, the embedding of position 2 in slots $d + 1, \dots, 2d$, and so on, the MLP head learns *position-specific* features. It can tell “the cat ate” from “ate the cat” because the embedding of each word lives in a different slice of $\mathbf{x}$.

This is a *hybrid* inductive bias. The embedding layer is position-equivariant (the same token has the same embedding regardless of position), but the MLP head is explicitly position-sensitive. Bengio gets word-identity invariance for free (one embedding per word) while still using word order through the MLP.

Other choices in the same design space:

- A **1-D CNN** (kernel size N , stride 1) shares weights across positions in the MLP head too: a single learned context detector applies at every position via convolution. Stronger prior, useful when patterns can appear anywhere in the sequence.
- **CBOW-style averaging** replaces concatenation with mean pooling over the embeddings: loses word order, gains full position invariance. Useful for retrieval, hopeless for language modeling.
- A **Transformer** uses attention over all N positions, with positional encoding added back in to recover sequence sensitivity.

Bengio’s choice (concatenate + MLP) is the simplest non-trivial point in this space. It hardwires positional sensitivity into the architecture without any conv or attention machinery.

4. Training

No BPTT and no recurrent unrolling. Each forward pass processes one N -token context and predicts one target token. Backward propagates from the loss back through the MLP, then through **EmbedConcat**, which splits the gradient on $\mathbf{x}$ into N chunks of size d and routes each back through the embedding in LIFO order (matching the embedding’s internal cache discipline).

```
net.fit(X, Y, epochs=4, lr=0.02, batch_size=1)
```

The existing **Network** training loop handles everything else. Each training example is independent: there is no inter-example state to carry forward, no `reset_cache()` calls scattered through the loop, no BPTT bookkeeping. The whole training script is shorter than the RNN’s.

A gradient check verifies the math: the standard `gradient_check` applied to a small **Network** built on **EmbedConcat** and the MLP head matches central finite differences to 10^{-4} relative error (`tests/test_gradients.py::test_gradient_embed_concat_network`).

5. The experiment: char-level Alice

Same corpus and vocabulary as the RNN post: the first 30,000 characters of *Alice’s Adventures in Wonderland*, 75-character vocabulary. Configuration:

- Context $N = 8$ chars (so each example is 8 ids in, one id out)
- Embedding dimension $d = 16$
- Hidden layer $h = 64$ units
- SGD with lr 0.02, batch size 1
- 4 epochs over the corpus

Total parameters:

$$75 \cdot 16 + 8 \cdot 16 \cdot 64 + 64 + 64 \cdot 75 + 75 = 14,331,$$

comparable to the RNN post’s 13,835.

The mean per-character loss starts at $\log 75 \approx 4.32$ (uniform random over the vocabulary) and descends to 2.04 nats per character (perplexity 7.67) over the 4 epochs. The RNN in the previous post landed at 2.06 over 15 epochs on the same corpus. The endpoints are essentially the same; the Bengio LM gets there in roughly a quarter of the epochs because there is no BPTT to unroll and

no recurrent dependencies between training examples. Every $(window, target)$ pair is independent, and stochastic gradient descent makes faster progress per step.

Generated samples. Seeded with "alice was ", sampled with temperature 0.8.

After 1 epoch (loss 2.57):

```
alice was it of tha bunding dass!  
ThI hat has fot ingh gh sathe casse of ch see fa  
fo han thind? fof she shis. be. las; the wans nat was a ca you sar  
tose caca hac th! "Mit caca the wand the fand ast mat, a th
```

After 2 epochs (loss 2.22):

```
alice was alain." "ase will krling all orl ut mrich ald copesal  
criyed of a canordond and whoce.
```

```
"Alice--t''s in eror a roulpsmald to meack, bet. cundt listlink dat  
hige not bugd roure tilk ap ily a gall ay hal
```

After 3 epochs (loss 2.11):

```
alice was oute alr toustereste kary he would beting ay see seal  
touker and could here sloagare hery alle way pathing af beeh, and  
ous outteall ige ouse allle betings hengend's cere of a sate soidund  
ssice toren
```

After 4 epochs (loss 2.04):

```
alice was op seer.
```

```
Thisu thipt in weam our now o gaulisg, on  
in cunbour, an, in with ounded on, now of aucking of hior, with a  
witt out-hat disttim siever wind "eried! Pit the sive wondsem--that  
pono it the sho
```

The trajectory has the same shape as the RNN's in the previous post: random letter frequencies, then recognizable short words and articles ("the", "was", "and", "could", "would"), then English-shaped fragments with occasional plausible phrases. Punctuation and quote balance are hopeless. Long-range coherence is absent. Character names other than "Alice" do not appear in the generated text in any reliable way.

The per-character cross-entropy in nats translates directly to bits per character: $2.04 / \ln 2 \approx 2.94$ bits per character. That is the model's compression rate on this corpus. The MDL identity is not an analogy; bits per character of cross-entropy *is* bits per character of a Shannon-optimal code under p_θ . A model that compresses better also generalizes better.

The full demo is `examples/fixed_context_lm.py`.

6. What Bengio gives up vs the RNN

The RNN's hidden state can in principle carry information from arbitrarily far back. Bengio's fixed context cannot. If the model needs to remember a token from position $t - 100$ to predict position t , the RNN has a chance and Bengio does not.

In practice, for char-level Alice at this scale, the difference is small. The vanilla RNN’s effective memory is also limited (the vanishing gradient discussed in the previous post limits it to perhaps a few dozen timesteps), and char-level prediction relies heavily on local patterns (morphology, word boundaries, common bigrams and trigrams). Both models capture these adequately.

For longer-range dependencies (multi-sentence narrative consistency, character names introduced hundreds of tokens ago) both models struggle. The Transformer post addresses the long-range case directly.

7. What Bengio gives up vs a 1-D CNN

A 1-D CNN with kernel size N would share the same weights across all positions in the MLP head: one learned context detector applied at every position via convolution. Bengio’s concatenation does the opposite: each position gets its own slice of the first hidden layer’s weights, so the model can learn “the immediately preceding token matters more than the one $N - 1$ back” or “subject-verb agreement is between slots 2 and 5.”

The trade is parameter efficiency vs position specificity. The CNN has fewer parameters (one kernel applied many times); the Bengio MLP has more (one parameter per (position, hidden unit) pair) but can specialize per position.

For language, position matters: the immediately preceding token is more informative than the one $N - 1$ back. So Bengio’s choice is well-suited even though it costs parameters. The 1-D CNN view is more natural for tasks where the same pattern can appear at any offset (motif detection in biological sequences, edge detection in 1-D signals).

8. What Bengio gains over the RNN

Three things:

- **No vanishing gradients.** Backward through the model is one MLP backward and N embedding backwards. No long Jacobian product over time. No spectral-radius issue.
- **Parallelism over examples.** Every (context, target) pair is independent of every other. Mini-batching is straightforward; sequence-level dependencies are gone.
- **Simpler training.** No BPTT, no `reset_cache()` between sequences. Each forward pass is independent of every other.

These are the same advantages that eventually made the Transformer attractive over the RNN: no recurrence, no vanishing gradients, easier parallelism. Bengio’s model got there first, for sequences short enough that fixed-context N is sufficient.

9. Where this sits in the inductive-bias map

Architecture	Position handling	Memory horizon
MLP	no sharing; flat input	full input dim
1-D CNN	weight sharing across positions	fixed (receptive field)
Bengio fixed-context LM	shared embedding + position-sensitive MLP	fixed window N
RNN	weight sharing across time; state carries	unbounded in principle

Architecture	Position handling	Memory horizon
Transformer	permutation-equivariant + positional code	full sequence

Bengio sits at a specific point: position-sensitive at the MLP level (unlike a 1-D CNN), with a hard memory cutoff at N tokens (unlike an RNN), and with no inter-position interaction beyond what the MLP head learns (unlike a Transformer, which has direct pairwise attention).

It is the architecture for problems where the relevant context is small and you know how small. When that assumption holds, it is fast, simple, and easy to train. When it does not, the unbounded-context architectures (RNN, Transformer) become necessary.

10. The Transformer is next

The Transformer is the natural answer to: “what if we kept Bengio’s no-recurrence-no-BPTT design but allowed every position to attend to every other, with weights learned from data rather than hardcoded by concatenation order?” That move replaces the position-sensitive MLP with a permutation-equivariant attention mechanism, then adds positional encoding back in to recover sequence sensitivity. The post is next.